

ARCHITECTURE DECISIONS

The following document explains the main infrastructure and architecture decisions for the LabResults API. It covers the reasoning behind the selected approaches across networking, compute, CI/CD, secret management, IAM, container security, and backup strategy, while also outlining the key assumptions, trade-offs, limitations, and potential future improvements associated with each decision.

1. Context & Assumptions

- Service: LabResults API, a backend API exposing laboratory results data.
- Team: Small (2 engineers), with no dedicated platform or SRE function.
- Budget: Low. Cost predictability is a strong requirement.
- Traffic: Simple web/API workload, sporadic, with no sustained high throughput. No batch, ML, or GPU workloads.
- Data sensitivity: Lab results imply PII and potentially health-adjacent data. Even without a stated compliance mandate, the data is treated as sensitive by default (encryption at rest and in transit, least privilege, and an audit trail), because these controls are relatively inexpensive to establish now and more expensive to retrofit later.
- No existing Kubernetes or multi-cloud investment. AWS is the target cloud for the foreseeable future.

2. Networking: 1 AZ vs 2 AZ

Approach: Deployment across two Availability Zones, with public and private subnets duplicated in each AZ, and a NAT Gateway in each AZ rather than one shared NAT.

Reasoning: A single AZ caps realistic availability at around 99.5–99.9%, which works out to roughly 9–44 hours of downtime a year. If that AZ has a problem, the whole service goes down because there is nowhere for traffic to move. A two-AZ deployment could provide around 99.95–99.99% availability, reducing potential downtime to roughly 1–4.4 hours a year, because the load balancer and ECS service can shift traffic to the healthy AZ automatically when one AZ degrades or goes offline.

For a lab results API, this matters more than it would for an internal tool. Other services or clinicians may depend on it for correctness and uptime, and an AZ-level outage taking down the entire API is a difficult failure mode to justify when the additional infrastructure cost is relatively low.

The NAT Gateway decision follows the same reasoning. A single shared NAT reduces some of the resilience gained from a two-AZ deployment. If the NAT in one AZ fails, the private subnet in the other AZ could lose internet egress even though its compute remains healthy. One NAT Gateway per AZ avoids that shared failure point.

Trade-offs:

- Two NAT Gateways cost about \$65–70/month combined, compared with \$32–35/month for one, plus per-GB processing charges either way.
- Compute costs increase if tasks are distributed across both AZs for redundancy.
- Cross-AZ traffic costs about \$0.01/GB in each direction for inter-AZ communication, which a single-AZ setup avoids.

Assumptions: The API is customer- or clinician-facing rather than purely an internal development tool, so downtime has a real cost. The additional monthly cost for a second NAT Gateway is considered acceptable within the budget.

Known limitations: Two AZs improve resilience but increase infrastructure cost. The decision depends on the acceptable balance between cost and downtime risk.

Future improvements: Multi-AZ RDS could be introduced once a database is added so the failover strategy also covers the data layer. Development and staging could use a single NAT Gateway to reduce cost, while production could retain two NAT Gateways.

3. Compute Platform: ECS Fargate vs EKS

Approach: The API would run on ECS Fargate.

Reasoning: With a small team and no in-house Kubernetes expertise, EKS's flat control-plane cost, together with node patching, add-on compatibility, and RBAC/IRSA management, introduces operational overhead for what is essentially an API backend. Fargate removes the node layer entirely, eliminating node maintenance and capacity planning, while task definitions remain simpler than Kubernetes manifests for a team handling both development and operations.

Trade-offs:

- Fargate has roughly a 20–30% per-vCPU/GB premium over EC2-backed compute.
- Cold starts of around 30–60 seconds are acceptable for this workload but would become more significant for latency-sensitive traffic.
- AWS-specific task definitions increase platform lock-in.

Assumptions: The team remains small, with no immediate need for custom schedulers, service mesh, or GPU workloads. AWS remains the only target platform.

Known limitations: Kubernetes-native features such as operators, DaemonSets, and service mesh are not available if future requirements depend on them.

Future improvements: The decision could be revisited if the team grows beyond roughly 10 engineers with a dedicated platform function, or if the service expands into dozens of microservices where Kubernetes-native traffic management becomes more valuable.

4. CI/CD Platform: GitHub Actions vs GitLab CI vs AWS CodePipeline + CodeBuild

Approach: GitHub Actions.

Reasoning: The source code is assumed to already live on GitHub, so GitHub Actions avoids introducing another platform, billing account, or workflow for developers already working there. It also provides mature OIDC support for AWS authentication without long-lived access keys, together with a large ecosystem of reusable actions for vulnerability scanning and ECS deployments.

GitLab CI would become more relevant if source code were already hosted on GitLab or if a single platform for source control, CI, and security scanning became a priority.

AWS CodePipeline and CodeBuild would provide a fully AWS-native workflow inside the IAM boundary, but would also introduce additional AWS services and configuration that are not currently required.

Trade-offs: Remaining on GitHub Actions keeps CI/CD tied to GitHub. A move toward AWS-native tooling could become relevant if consolidated AWS governance becomes a future priority.

Assumptions: The repository is hosted on GitHub, the team is already familiar with GitHub workflows, and there is a single AWS account without a complex promotion pipeline.

Known limitations: GitHub Actions provides fast operations, but dedicated enterprise CI/CD platforms can offer deeper control over build agents, networking, deployment environments, and integrations.

Future improvements: GitHub Environments could provide deployment protections by associating deployments with named environments such as production. Manual approval gates, protected branches, release tags, and concurrency controls could also be introduced.

5. Secret Management

Approach: AWS Secrets Manager for rotating secrets such as database credentials and API keys, and SSM Parameter Store (Standard tier) for low-sensitivity configuration.

Reasoning: Secrets Manager supports automatic rotation, while Parameter Store Standard is free and sufficient for configuration values that do not require rotation. Splitting responsibilities between the two services keeps costs lower without sacrificing functionality.

Trade-offs: Maintaining two configuration stores introduces some classification overhead, although consistent naming and tagging could reduce it.

Assumptions: A relational database will eventually be introduced, requiring credential rotation, while the overall number of secrets remains relatively small.

Known limitations: Neither Secrets Manager nor Parameter Store resources currently exist in the Terraform configuration, and non-database secrets would initially rely on manual rotation.

Future improvements: Automatic rotation could extend beyond database credentials. Secret-scanning with tools such as Gitleaks could also become part of the CI pipeline, and cross-region replication could become relevant if disaster recovery requirements expand.

6. IAM Governance

Approach: Purpose-specific IAM roles rather than broad administrative roles.

- `github_actions_ci`: CI/CD role with OIDC trust restricted to refs/heads/main, scoped ECR permissions, and tightly controlled PassRole permissions.
- `ecs_execution`: Image pull and log push only.
- `ecs_task`: Runtime permissions that begin empty and expand only as application requirements grow.

Reasoning: Resource-scoped IAM policies require more upfront work, but the cost is paid once and provides stronger protection for sensitive data.

Trade-offs: More granular policies increase initial implementation effort compared with broad administrator permissions.

Assumptions: A single AWS account is used, and Terraform pull-request reviews provide sufficient governance for a small team.

Known limitations: Permission boundaries and Access Analyzer are not yet present in the Terraform configuration.

Future improvements: AWS Organizations, separate production and non-production accounts, Service Control Policies, and formal access reviews could become appropriate as compliance and team size increase.

7. Container Security

Approach: Layered controls, most of which are already reflected in the pipeline and Terraform.

1. Minimal base images reduce attack surface.
2. Immutable image tags prevent silent tag replacement.
3. ECR image scanning and Trivy provide vulnerability visibility and release gating.
4. Registry encryption protects images at rest.
5. Fargate provides runtime isolation.
6. Least-privilege task roles limit application permissions.

Reasoning: The combination of registry scanning, runtime isolation, immutable artifacts, and least-privilege access provides a practical baseline for a small production workload without introducing unnecessary operational complexity.

Trade-offs: Blocking releases on critical vulnerabilities can delay deployments, although ignore-unfixed: true limits blocking to vulnerabilities with available fixes.

Assumptions: Dockerfiles already use minimal or distroless base images.

Known limitations: Runtime threat detection and policy enforcement are not explicitly configured beyond Fargate's built-in isolation.

Future improvements: GuardDuty ECS/Fargate runtime monitoring and image signing with Cosign could strengthen runtime and supply-chain security.

8. Backup Strategy

Approach: The current Terraform configuration contains no stateful data store, so persistence is not yet part of the deployed infrastructure.

A future database design could include:

- RDS PostgreSQL with daily automated snapshots and 7–14 days of retention.
- Point-in-time recovery.
- AWS Backup for future EFS storage.
- Encrypted backups using the same key strategy as the rest of the environment.

Reasoning: Point-in-time recovery provides significant value for sensitive data while keeping storage costs manageable through relatively short retention periods.

Trade-offs: Longer retention and point-in-time recovery increase storage costs. Multi-AZ RDS also increases database costs substantially, although it addresses availability rather than backup itself.

Assumptions: Lab results data will require recovery capabilities, but cross-region disaster recovery is not yet necessary.

Known limitations: No backup infrastructure currently exists in Terraform.

Future improvements: Multi-AZ RDS, cross-region snapshot copies, and periodic restore testing could become appropriate once production traffic and operational requirements increase.